

13. Cryptographic Constructions & Symmetry Constraints

This section studies how symmetry constraints shape cryptographic design and how they can be used, carefully, in cryptanalysis. The central theme is that many security-relevant properties are not merely algebraic accidents of a particular implementation, but are tied to invariants under affine transformations, round composition, and group actions on the underlying state space.

A useful starting point is the avalanche behavior of a round function or full cipher. For a cipher family indexed by a round parameter r , one may summarize diffusion by an error profile $\varepsilon(r)$, interpreted as a measure of how strongly small input perturbations spread through the state after r rounds. In practice, this is often examined alongside differential and linear bias: differential behavior tracks how input differences propagate to output differences, while linear bias measures correlations between affine masks on input and output. These quantities are not independent of symmetry. If a design admits large automorphism groups or repeated structural patterns, then some difference or mask classes may be preserved or mapped into one another, producing regularities that are visible in reduced-round experiments. In that sense, $\varepsilon(r)$ is not just a performance summary; it is also a probe for hidden invariants that survive partial composition.

S-boxes provide the most concentrated setting for symmetry analysis. Two S-boxes that are affine equivalent differ only by invertible affine changes of coordinates on the input and output, while CCZ equivalence is a broader relation that preserves several cryptographic invariants of interest. Under these equivalences, properties such as nonlinearity and differential uniformity are central. High nonlinearity helps suppress linear approximation, while low differential uniformity limits the number of solutions to difference equations and is therefore desirable for resistance to differential attacks. The symmetry viewpoint is useful here because it separates intrinsic cryptographic quality from superficial coordinate choices: an S-box may look different after a change of basis, yet remain in the same equivalence class with the same essential resistance profile. Conversely, two S-boxes that are not equivalent may still share some coarse metrics, so equivalence classes should be treated as a structural lens rather than a complete security certificate.

Round-based constructions such as SPN and Feistel networks make these issues more subtle. In an SPN, repeated application of substitution, permutation, and key mixing can amplify local S-box properties into global diffusion, but the composition also introduces invariants that may survive across rounds. In a Feistel structure, the split state and alternating update rule impose a different symmetry pattern, often yielding useful algebraic regularities for analysis. In both cases, one studies which quantities remain stable under round composition, which are merely permuted, and which are destroyed by the mixing layer. This is especially important when comparing full-round security claims with reduced-round experiments: a property that appears strong at one round count may be an artifact of a surviving symmetry rather than a robust indicator of long-term behavior. The practical lesson is that round count, key schedule, and state representation must be reported together, because each can change which invariants are visible.

Group actions provide a unifying language for these design and analysis questions. A cryptographic primitive can be acted on by affine coordinate changes, bit permutations, state relabelings, or more specialized equivalence relations, and the resulting orbits organize families of functionally related designs. From the design side, one often seeks to constrain symmetry so that the construction is not overly rigid; excessive symmetry can create exploitable structure. From the cryptanalysis side, one may exploit symmetry to reduce search spaces, classify equivalent instances, or identify invariant subspaces and fixed points. The same formalism also clarifies closure pitfalls: a class of constructions may be closed under a natural operation such as composition or conjugation, yet that closure can hide degenerate members whose apparent complexity does not translate into security. Closure under a transformation should therefore be read as a statement about membership in a

family, not as a guarantee that every member is equally robust.

An important methodological caution is that symmetry-preserving transformations are not automatically security-preserving in the operational sense. Equivalence classes can preserve nonlinearity or differential uniformity while still changing implementation cost, side-channel exposure, or the interaction with a particular key schedule. Likewise, a family closed under a chosen group action may still contain weak representatives if the action is too coarse to distinguish them. For this reason, symmetry analysis should be paired with explicit metrics, round-reduced experiments, and implementation-aware checks rather than treated as a substitute for them. In particular, one should distinguish mathematical invariants from engineering invariants: the former may survive coordinate changes, while the latter depend on hardware, software, and leakage model.

The section therefore supports an artifact-driven workflow. An equivariance classifier can be used to identify which transformations preserve the relevant design class, and round-reduced experiments can probe how $\varepsilon(r)$, differential bias, and linear bias evolve as rounds are added. Such experiments are most informative when reported with the exact equivalence relation, the round count, and the state representation used in the test harness. A useful artifact package would include the classifier’s decision rule, the canonicalization procedure used to compare instances, and a small suite of benchmark ciphers or S-box families for which the invariants are known. In the spirit of the book, the goal is not to claim security from symmetry alone, but to use symmetry as a disciplined lens for organizing constructions, pruning redundant cases, and exposing structural weaknesses.

Ethically, these tools should be used to improve robustness and to understand failure modes, not to facilitate misuse. The same methods that classify equivalent designs and accelerate cryptanalysis can also help designers avoid accidental structural weaknesses and can support reproducible evaluation of candidate primitives. When shared responsibly, symmetry-based analysis becomes a way to make cryptographic evaluation more transparent, more comparable across implementations, and less dependent on informal intuition.